
Group 12

Group 12

Software Architecture Document

For SinhaSpeech

Version 1.0

Project ID : P13

Group ID : 12

Team Members:

230283A Yohan Jayasinghe

230295L Tharupahan Jayawardana

230277J Imindu Jayasekara

Mentor: Dr. Buddhika Karunaratna

Teaching Assistant: Mr. Ilampoornan

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

Revision History

Date	Version	Description	Author
30-Jul-2026	1.0	Initial architecture document	Group 12

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

Table of Contents

1. Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Definitions, Acronyms, and Abbreviations	4
1.4 References	5
1.5 Overview	6
2. Architectural Representation	6
3. Architectural Goals and Constraints	7
3.1 Architectural Goals	7
3.2 Architectural Constraints	8
4. Use-Case View	9
4.1 Use-Case Realizations	10
5. Logical View	11
5.1 Overview	11
5.2 Architecturally Significant Design Packages	11
6. Process View	13
Phase 1 - Offline training processes	13
Phase 2 - Online request-handling processes	15
7. Deployment View	19
8. Implementation View	21
8.1 Overview	21
8.2 Layers	21
9. Data View	22
10. Size and Performance	25
11. Quality	25

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

1. Introduction

This Software Architecture Document (SAD) provides a comprehensive architectural overview of SinhaSpeech, a Sinhala speech-to-text ASR system. It captures and conveys the significant architectural decisions made across the fine-tuning and evaluation pipeline (Phase 1) and the accessibility application built on top of it (Phase 2), using a number of complementary architectural views.

1.1 Purpose

This document provides a comprehensive architectural overview of the SinhaSpeech system, using a number of different architectural views to depict different aspects of the system: the Use-Case View, Logical View, Process View, Deployment View, Implementation View, and Data View. It is intended to capture and convey the significant architectural decisions made on the system, including the separation between the Phase 1 model pipeline and the Phase 2 accessibility application, and the layered structure of the Phase 2 system (Presentation, Application, AI, and Data layers).

The intended audience for this document includes the project mentor and teaching assistant, who use it to evaluate the soundness of the system design; the development team, who use it as a shared reference during implementation; and any future contributor who needs to understand how the system is structured before extending it.

1.2 Scope

This document applies to the full SinhaSpeech system as defined in the project's Software Requirements Specification (SRS): the Phase 1 fine-tuning and evaluation pipeline, and the Phase 2 accessibility application comprising the Presentation Layer (web application), Application Layer (backend API), AI Layer (fine-tuned Whisper model), and Data Layer (databases and object storage). It does not cover the internal architecture of third-party components (e.g., the internals of the Whisper model itself, or of Weights & Biases), which are treated as external dependencies.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
ASR	Automatic Speech Recognition.
WER	Word Error Rate, the standard word-level ASR accuracy metric.
CER	Character Error Rate, an ASR accuracy metric computed at the character level.
VAD	Voice Activity Detection.
Whisper	OpenAI's multilingual speech recognition model family, used as the base model for fine-tuning.
SpecAugment	A data augmentation technique masking portions of a spectrogram during training.

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

Term	Definition
DHH	Deaf and hard-of-hearing.
Code-mixing	Interchangeable use of Sinhala and English within the same utterance.
Checkpoint	A saved snapshot of a model's trained parameters.
W&B	Weights & Biases, the experiment-tracking platform used for this project.
FP16	Half precision (mixed precision) training, used to reduce memory and speed up training.
REST	Representational State Transfer, the API design style used by the Phase 2 backend.
JWT	JSON Web Token, used for authenticated session handling between the front end and backend.

1.4 References

- [1] SinhaSpeech Feasibility Study, Group 12, 26 July 2026.
- [2] SinhaSpeech Project Proposal, Group 12, 26 July 2026.
- [3] SinhaSpeech Software Requirements Specification, Version 1.0, Group 12.
- [4] O. Kjartansson, S. Sarin, K. Pipatsrisawat, M. Jansche, and L. Ha, "Crowd-Sourced Speech Corpora for Javanese, Sundanese, Sinhala, Nepali, and Bangladeshi Bengali," in Proc. 6th Workshop on Spoken Language Technologies for Under-Resourced Languages (SLTU 2018), 2018.
- [5] OpenSLR, "SLR52 – Large Sinhala ASR Training Data Set." Available: openslr.org/52 (Accessed on 25-Jul-2026).
- [6] Mozilla Common Voice – Sinhala subset. Available: mozilladatacollective.com/datasets/ (Accessed on 25-Jul-2026).
- [7] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)," OpenAI, 2022.
- [8] Hugging Face, "Fine-Tune Whisper for Multilingual ASR with Transformers." Available: huggingface.co/blog/fine-tune-whisper (Accessed on 25-Jul-2026).
- [9] SPEAK-ASR Hugging Face organization. Available: huggingface.co/SPEAK-ASR (Accessed on 25-Jul-2026).
- [10] Weights & Biases (W&B) documentation. Available: docs.wandb.ai (Accessed on 25-Jul-2026).
- [11] mermaid.js documentation, used to draw the diagrams in this document. Available: mermaid.js.org (Accessed on 26-Jul-2026).
- [12] draw.io / diagrams.net, used to render and export the diagrams in this document. Available: app.diagrams.net (Accessed on 26-Jul-2026).

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

1.5 Overview

The remainder of this document is organized as follows. Section 2 describes how the system's architecture is represented. Section 3 describes the architectural goals and constraints. Section 4 presents the Use-Case View. Section 5 presents the Logical View. Section 6 presents the Process View. Section 7 presents the Deployment View. Section 8 presents the Implementation View. Section 9 presents the Data View. Section 10 discusses size and performance. Section 11 discusses quality attributes. Section 12 lists references.

2. Architectural Representation

SinhaSpeech follows a layered, modular architecture designed around a clean separation between model development and application delivery. At the foundation, the Phase 1 pipeline unifies multiple Sinhala speech datasets OpenSLR52, Mozilla Common Voice Sinhala, YouTube-Sinhala-ASR, and Biz-Brains-Academy-Sinhala through a standardised preprocessing pipeline that handles audio normalisation, silence trimming, noise reduction, and transcript cleaning before feeding the base multilingual Whisper model into fine-tuning. This produces a benchmarked Sinhala ASR checkpoint, evaluated for accuracy and generalisation using WER and CER, that becomes the foundation for everything built on top of it.

Built on that checkpoint, the Phase 2 accessibility application is structured into four layers: a Presentation Layer (a web application providing lecture captioning, spoken-answer transcription, and self-study note-taking screens), an Application Layer (a backend API coordinating authentication, upload/recording, speech processing, confidence analysis, transcript management, and export), an AI Layer (the fine-tuned Whisper model, exposed as an inference service), and a Data Layer (a database and object storage holding transcripts, recordings, and user data). Requests flow in one direction, from the browser through the backend to the model and back, with the Data Layer accessed only through Application Layer services, keeping storage decoupled from presentation.

This architecture is represented using five complementary views, consistent with the 4+1 view model. The Use-Case View describes the system's functionality from the perspective of students, teachers, and, for Phase 1, the development team. The Logical View describes the system's decomposition into layers and components: Presentation, Application, AI, and Data. The Process View describes the system's runtime behaviour, distinguishing the offline Phase 1 training pipeline from the online, asynchronous audio-to-transcript processing pipeline in Phase 2. The Deployment View describes the physical nodes on which the system runs, from the student or teacher's browser through to the cloud-hosted web server, backend, model, and storage. The Implementation View describes the layered component structure shown in the system architecture diagram (Figure 1) and the responsibilities of each layer. A Data View is also included, given that persistent storage of classroom audio, transcripts, and user data is architecturally significant to this system, particularly with respect to the data-privacy requirements defined in the SRS.

This separation of concerns keeps data flow clear, from raw speech data through fine-tuning and evaluation to a deployed, accessible application, while allowing the model to be improved independently of the application layer built around it.

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

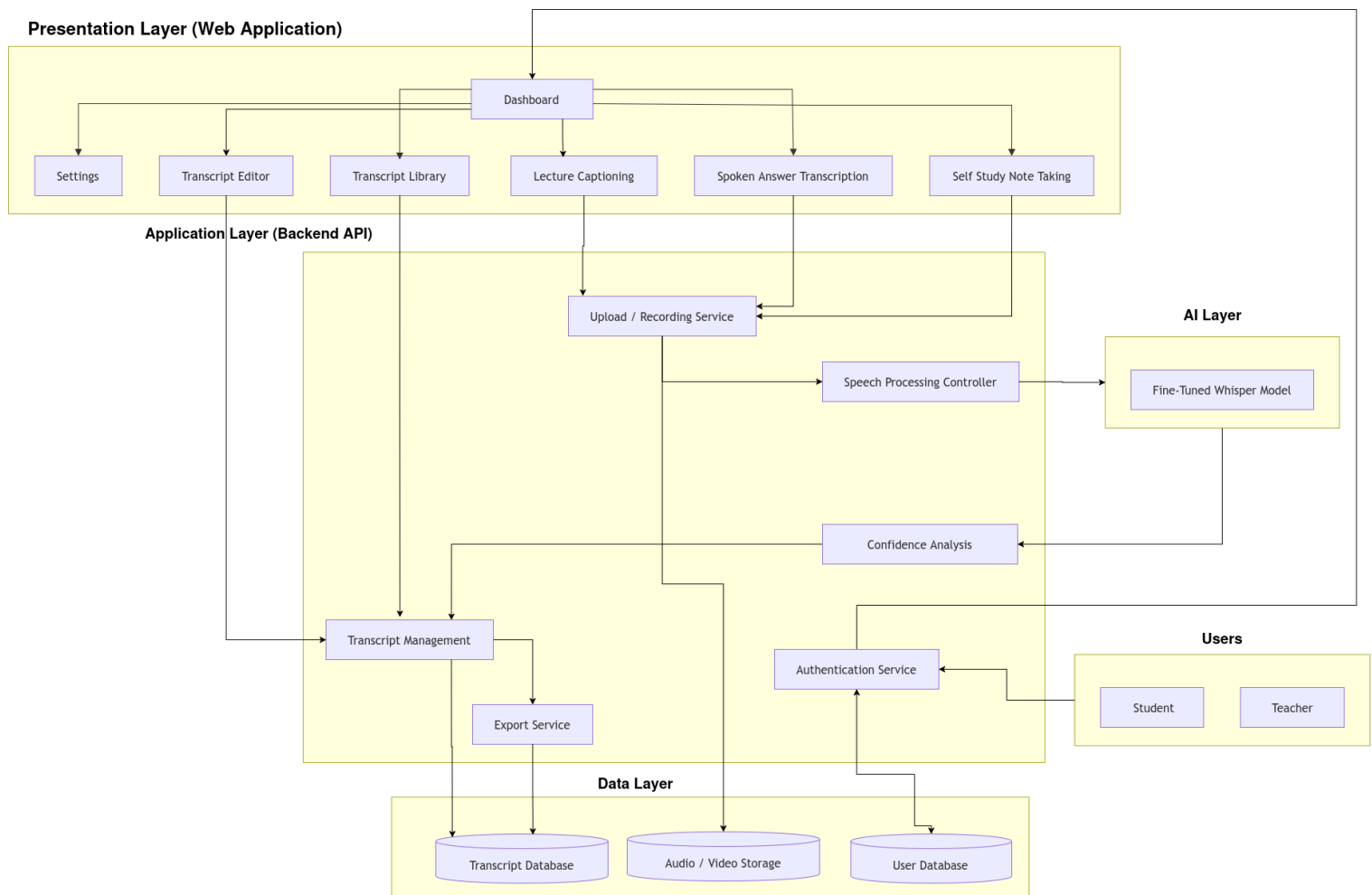


Figure 1 : Overall Architecture Diagram

3. Architectural Goals and Constraints

3.1 Architectural Goals

- **Modularity** - Preprocessing, fine-tuning, evaluation, and the Phase 2 application (front end, inference API, database) are structured as independent, replaceable components.
- **Accuracy-first design** - The architecture keeps the Phase 2 application entirely dependent on, but decoupled from, the Phase 1 model checkpoint, so the application can be built and tested without re-running fine-tuning, and the model can be improved without changing the application layer.

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

- Accessibility - The Presentation Layer is designed around the needs of Deaf/hard-of-hearing students and students with writing difficulties: adjustable font size, high contrast, low-friction inline editing, and confidence highlighting.
- Reproducibility - Every architectural decision in the Phase 1 pipeline (data split, model configuration, evaluation protocol) is documented so it can be independently reproduced.

3.2 Architectural Constraints

- Whisper's feature extractor requires 16kHz, mono, WAV audio input, which fixes the target format of the preprocessing component regardless of the original format of each source dataset.
- Available GPU memory and session limits on the team's cloud platform (Google Colab Pro, with free-tier fallbacks) constrain model choice, keeping Whisper-Base as an architectural fallback to Whisper-Small.
- Dataset licensing restricts the system to the open-research terms under which OpenSLR52, Mozilla Common Voice Sinhala, YouTube-Sinhala-ASR, and Biz-Brains-Academy-Sinhala are published.
- Sri Lanka's Personal Data Protection Act constrains how classroom audio, transcripts, and student data are stored and accessed in the Data Layer.
- The academic semester timeline bounds the overall schedule; the Phase 1 pipeline is prioritised as core, non-negotiable scope, with the Phase 2 application layered on top once the model clears its accuracy threshold.

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

4. Use-Case View

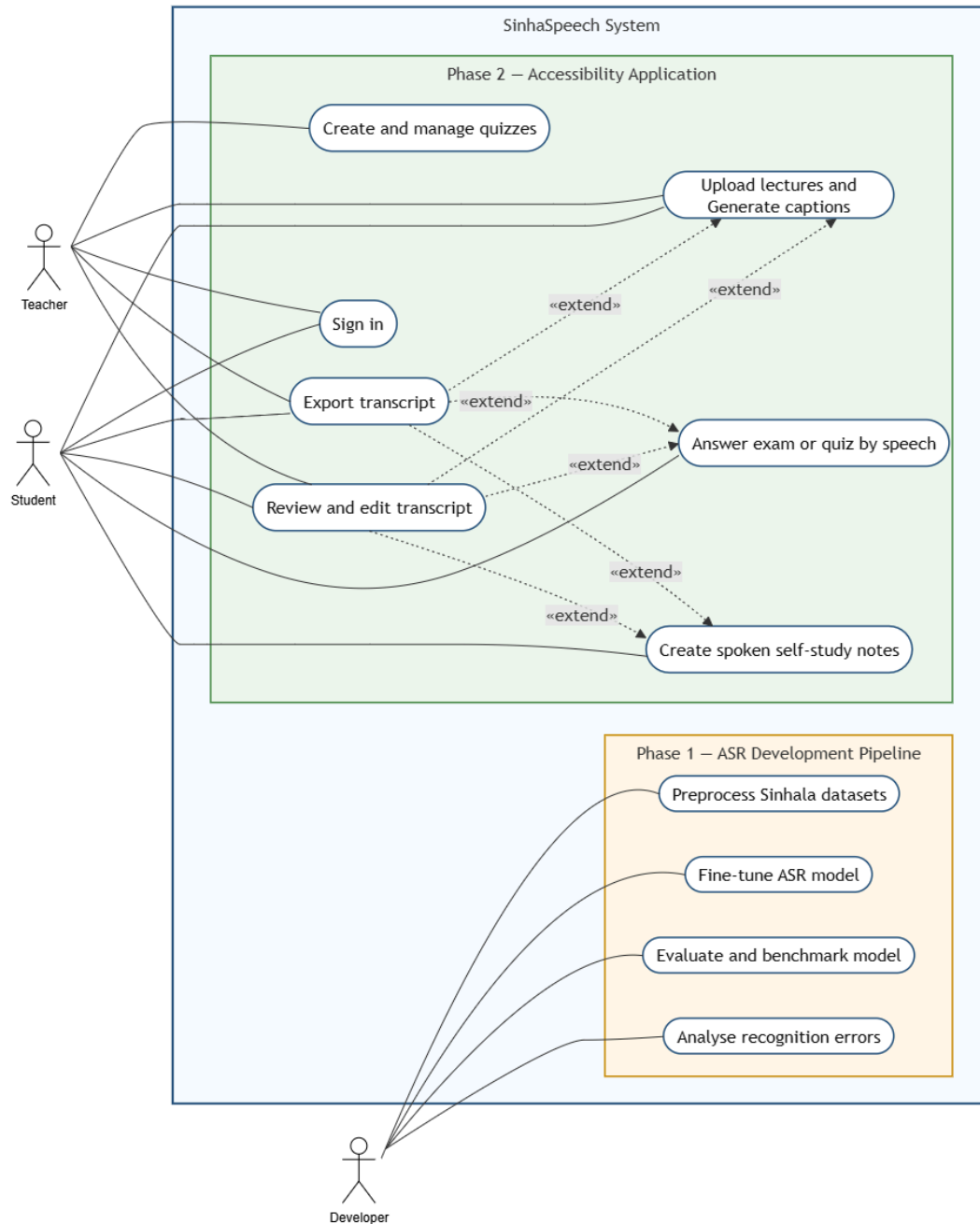


Figure 2 : Use case Diagram

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

Figure 2 shows the use cases and their actors. UC-1 through UC-7 represent the Phase 2 accessibility application used by students and teachers, while UC-8 through UC-11 represent the Phase 1 ASR development pipeline performed by the development team. Signing in is modelled as a separate use case and acts as a precondition for accessing the protected Phase 2 functionality. UC-5, Review and Edit Transcript, and UC-6, Export Transcript, extend the lecture-captioning, spoken-answer, and self-study note-taking workflows because these actions occur optionally after a transcript has been generated. Phase 1 covers dataset preprocessing, model fine-tuning, benchmarking, and recognition-error analysis.

4.1 Use-Case Realizations

The following use cases represent the significant, architecturally relevant functionality of SinhaSpeech, spanning both end-user-facing (Phase 2) and development-facing (Phase 1) functionality.

ID	Use Case	Primary Actor	Description
UC-1	Sign in	Student / Teacher	Authenticate the user and provide access to features permitted for their role.
UC-2	Upload lectures and generate captioning	Teacher / Student	Upload an audio or video lecture and receive a time-stamped transcript.
UC-3	Answer exam/quiz by speech	Student	Record or upload a spoken answer to a teacher-assigned or self-study exam/quiz question.
UC-4	Record self-study notes	Student	Record spoken notes and receive an editable transcript for later study.
UC-5	Review and edit transcript	Student / Teacher	Review a generated transcript, correct low-confidence words, and save changes.
UC-6	Export transcript	Student / Teacher	Export a finalised transcript as .txt, .docx, or .pdf.
UC-7	Create/manage quiz	Teacher	Create quiz or exam questions available to students through the platform.
UC-8	Preprocess Sinhala datasets	Development Team	Prepare the datasets for model training.
UC-9	Fine-tune ASR model	Developer	Run a fine-tuning configuration on preprocessed Sinhala data and log results to W&B.
UC-10	Evaluate and benchmark model	Developer	Compute WER/CER for the fine-tuned model against base Whisper and existing Sinhala fine-tunes.
UC-11	Analyse recognition errors	Development Team	Examine common transcription errors and confusion patterns

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

5. Logical View

5.1 Overview

The Logical View describes the main classes and services of the SinhaSpeech web application. The WebApplication allows users to sign in, upload or record media, manage transcripts, and use quizzes.

The application separates its main functions into services. The AuthenticationService manages user access, the MediaService manages audio and video files, and the TranscriptionService uses the fine-tuned WhisperModel to convert Sinhala speech into text. The TranscriptService handles transcript editing and exporting, while the QuizService manages quizzes, questions, and spoken answers.

The main data classes are User, MediaFile, Transcript, Quiz, Question, and AnswerSubmission. Although layers are not shown directly in the class diagram, the classes have separate responsibilities, making the system easier to develop, test, and maintain.

5.2 Architecturally Significant Design Packages

Web Application Package

Contains the WebApplication class. It provides the user interface and sends user requests to the correct service.

Authentication Package

Contains the AuthenticationService and User class. It handles sign-in, sign-out, and user authorization.

Media Management Package

Contains the MediaService and MediaFile class. It validates, stores, and retrieves uploaded or recorded media.

Transcription Package

Contains the TranscriptionService and WhisperModel. It converts Sinhala speech into text and checks the confidence of the generated transcript.

Transcript Management Package

Contains the TranscriptService and Transcript class. It supports viewing, editing, finalizing, and exporting transcripts.

Quiz Management Package

Contains the QuizService, Quiz, Question, and AnswerSubmission classes. It manages quizzes, questions, and students' spoken-answer submissions.

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

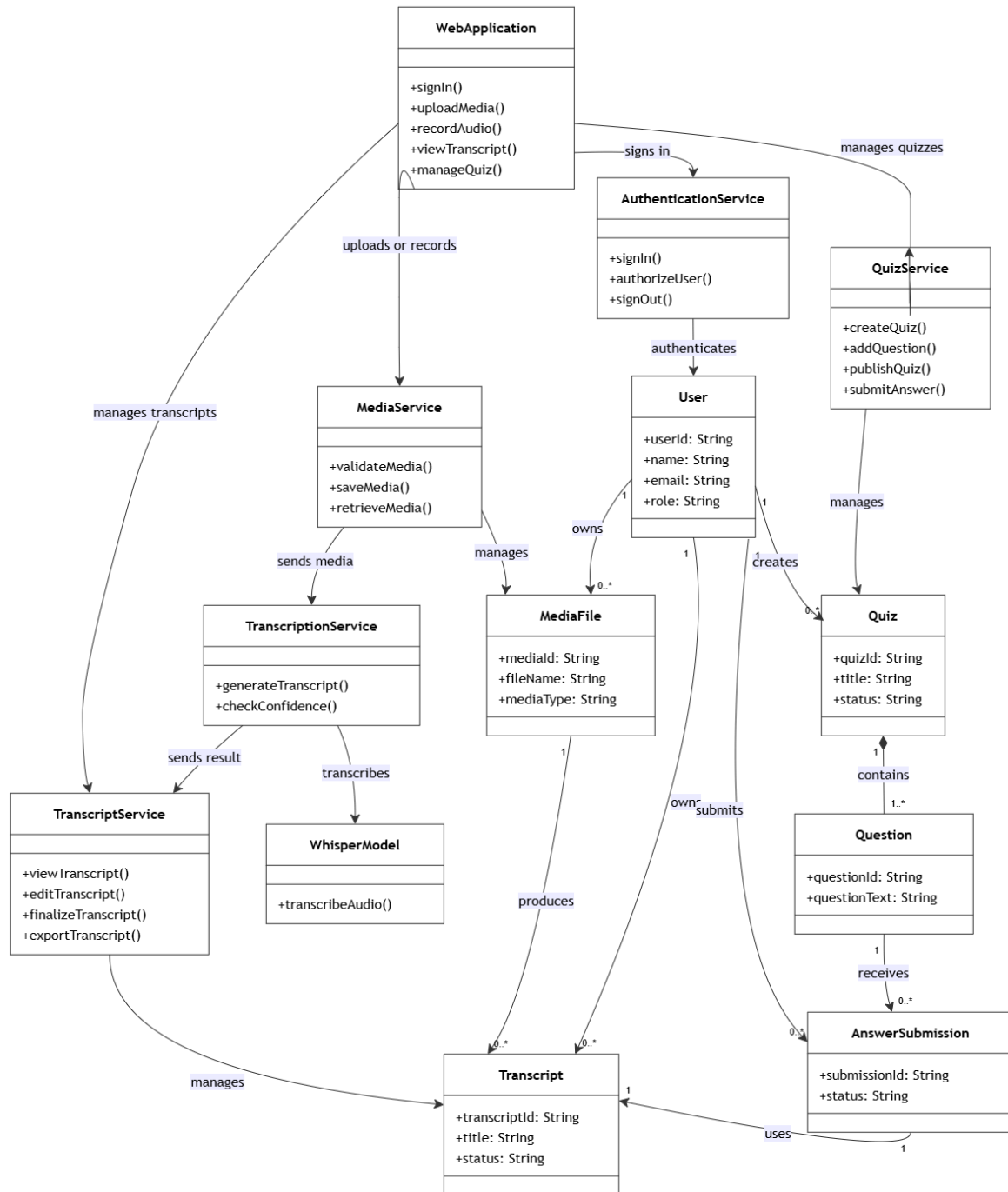


Figure 3 : Class diagram

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

6. Process View

SinhaSpeech has two largely independent process groupings, reflecting the two-phase project structure.

Phase 1 - Offline training processes

Preprocessing, fine-tuning, and evaluation run as separate, sequential batch processes on the team's cloud GPU platform (Google Colab Pro). These are not part of the live application's runtime and communicate with Weights & Biases over HTTPS for experiment logging. Each of the fine-tuning configurations runs as an independent process, allowing the three team members to work in parallel.

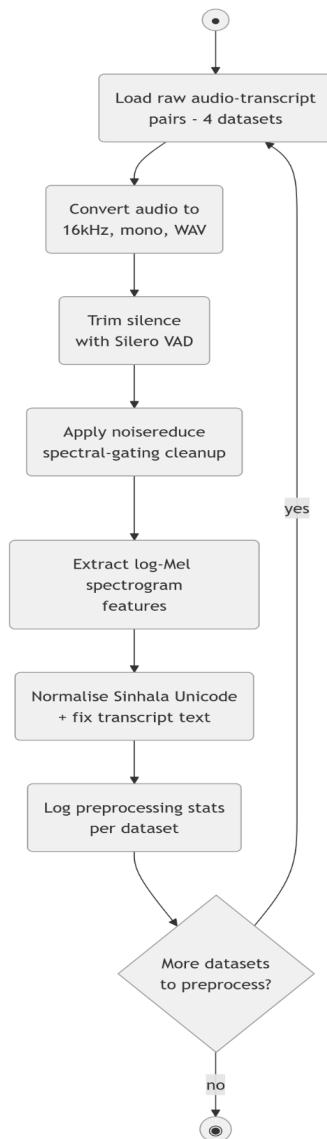


Figure 4 : Activity Diagram for Preprocessing pipeline

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

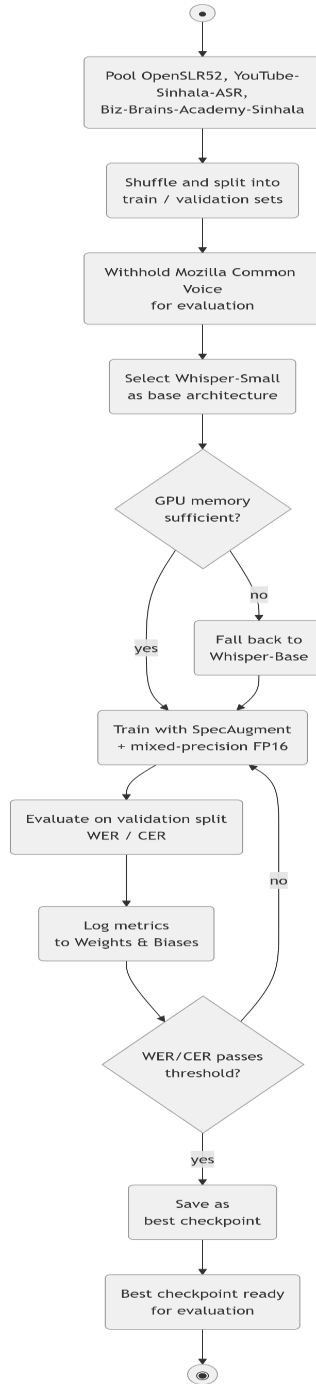


Figure 5 : Activity Diagram for finetuning workflow

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

Phase 2 - Online request-handling processes

The backend API handles each incoming HTTP request (e.g., an upload, a login, a transcript edit) as a lightweight, stateless process. Because speech transcription is comparatively long-running relative to a typical web request, the Speech Processing Controller processes transcription asynchronously: the client receives an acknowledgement immediately and polls for or is notified of the completed transcript, keeping the user interface responsive per the Task Efficiency usability requirement in the SRS.

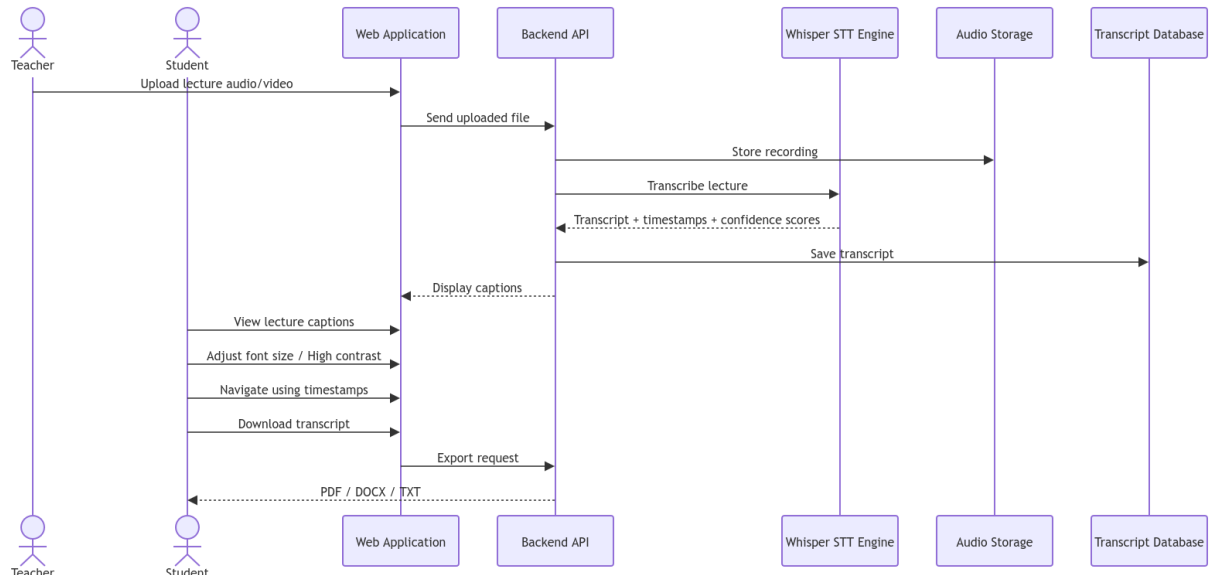


Figure 6 : Video / Audio Transcription Sequence Diagram

This sequence diagram illustrates the Lecture Captioning workflow for Deaf and hard-of-hearing students. A teacher or a student upload lecture audio or video through the Web Application, which forwards the file to the Backend API. The Backend API stores the recording in Audio Storage and sends it to the Whisper STT Engine for transcription. Once the engine returns the transcript along with timestamps and confidence scores, the Backend API saves it to the Transcript Database and displays the captions back through the Web Application. The student can then view the lecture captions, adjust font size and contrast, navigate using timestamps, and download the finished transcript, with the Backend API handling the export request and returning the file in the student's chosen format (PDF, DOCX, or TXT).

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

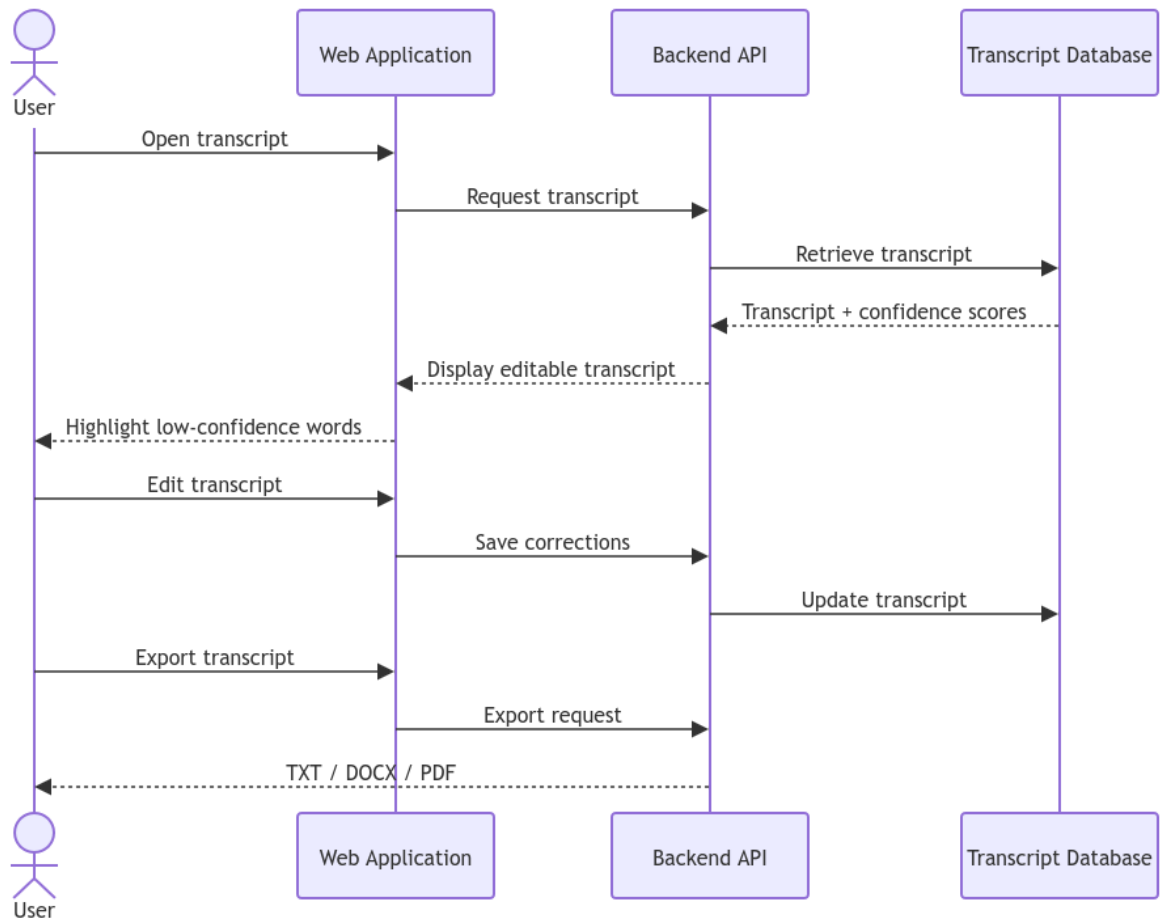


Figure 7 : Transcript Editing, Export, and Confidence Highlighting Sequence Diagram

This sequence diagram illustrates the shared transcript-editing workflow used across the platform. A user opens a transcript, which the Web Application retrieves from the Backend API and, in turn, from the Transcript Database, along with its confidence scores. The transcript is displayed in an editable view with low-confidence words highlighted, guiding the user's review. As the user makes corrections, the Web Application saves them through the Backend API to the Transcript Database. Finally, the user can export the corrected transcript, with the Backend API returning the file in the chosen format (TXT, DOCX, or PDF).

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

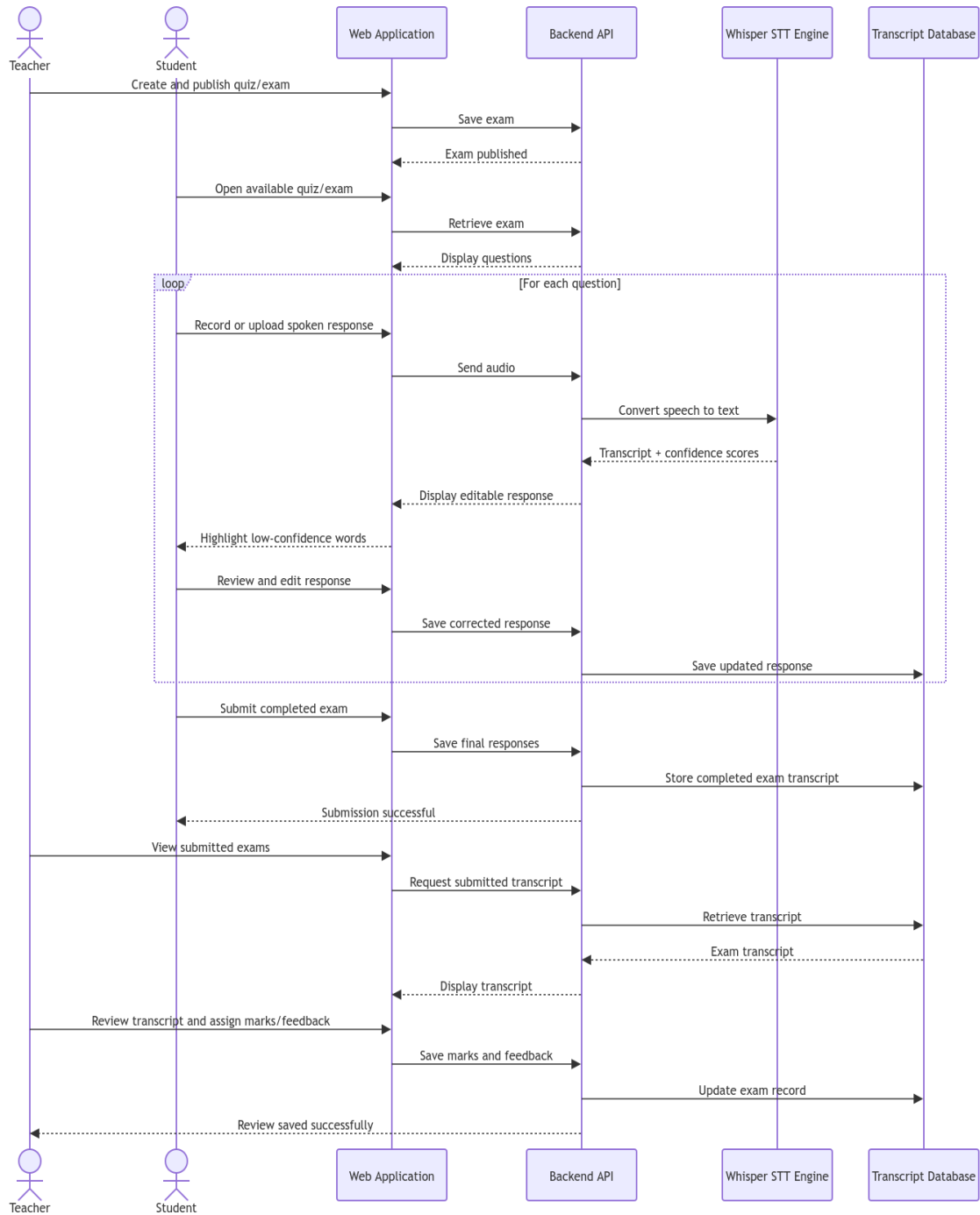


Figure 8 : Spoken Answer Transcription Sequence Diagram

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

This sequence diagram illustrates the exam/quiz answering workflow shared between teachers and students. A teacher creates and publishes a quiz or exam, which is saved through the Backend API. A student then opens the available quiz and, for each question, records or uploads a spoken response; the audio is converted to text by the Whisper STT Engine and returned with confidence scores, letting the student review and correct low-confidence words before the response is saved. This loop repeats for every question in the exam. Once all questions are answered, the student submits the completed exam, which is stored as a final transcript in the Transcript Database. The teacher can then retrieve the submitted exam, review the transcript, and assign marks or feedback, which is saved back to the exam record.

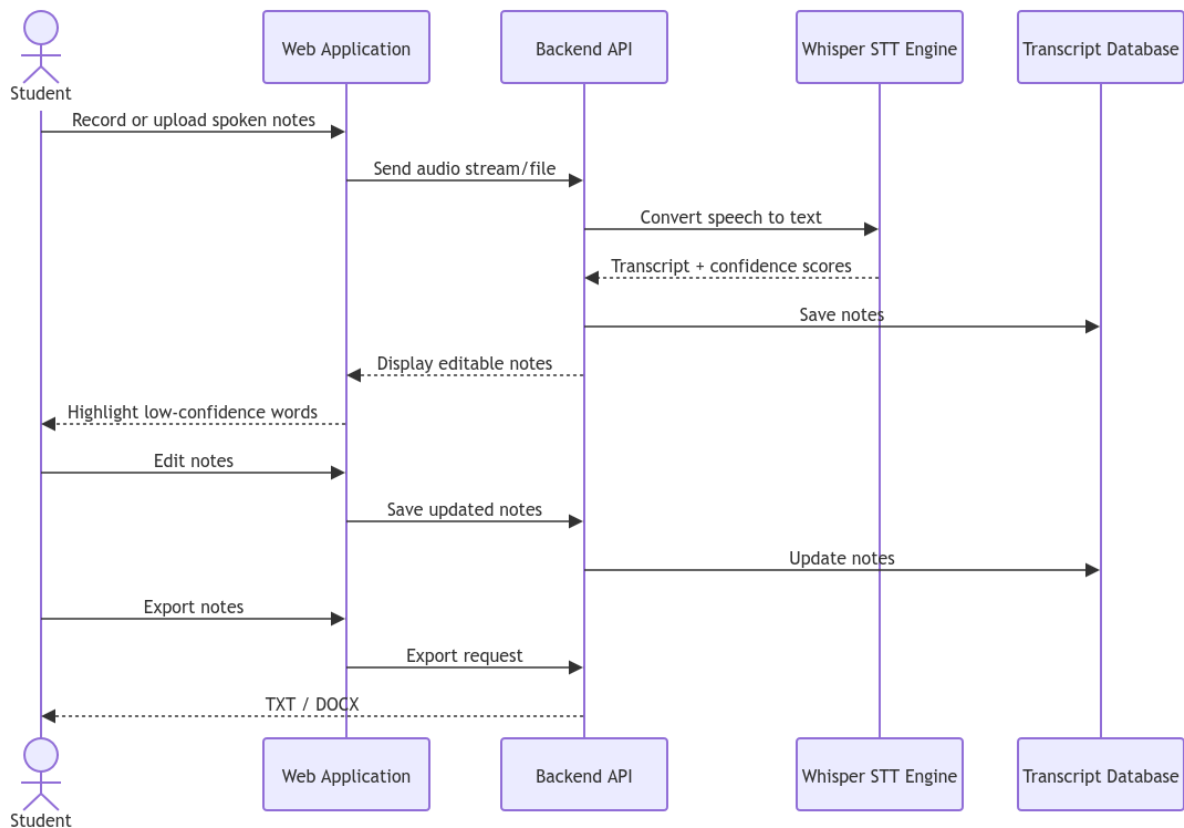


Figure 9 : Self-Study Note-Taking Sequence Diagram

This sequence diagram illustrates the self-study note-taking workflow. A student records or uploads spoken notes, which the Backend API sends to the Whisper STT Engine for transcription. The resulting text and confidence scores are returned and saved to the Transcript Database, then displayed to the student as an editable transcript with low-confidence words highlighted. The student can edit the notes, with corrections saved back through the Backend API, and finally export the finished notes as a TXT or DOCX file.

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

7. Deployment View

The system is deployed across the following physical/logical nodes:

- **Client device** - a student or teacher's laptop or desktop browser, communicating over standard broadband or classroom Wi-Fi.
- **Web application server** - hosts the Presentation Layer (React/Angular/Vue front end), served over HTTPS, deployed on a free-tier or low-cost host (e.g., Hugging Face Spaces, Render, or Railway).
- **Backend API server** - hosts the Application Layer (FastAPI/Flask), exposing REST/JSON endpoints, co-located with or adjacent to the AI Layer for low-latency inference calls.
- **Model inference node** - serves the Fine-Tuned Whisper Model, invoked synchronously by the Speech Processing Controller.
- **Database server** - hosts the three persistent stores in the Data Layer (Transcript Database, Audio/Video Storage, User Database), accessed over a secure, authenticated connection.
- **Training environment (Phase 1 only, offline)** - Google Colab Pro or an equivalent cloud GPU platform, entirely separate from the Phase 2 production deployment, used only to produce the model checkpoint that is later deployed to the model inference node.

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

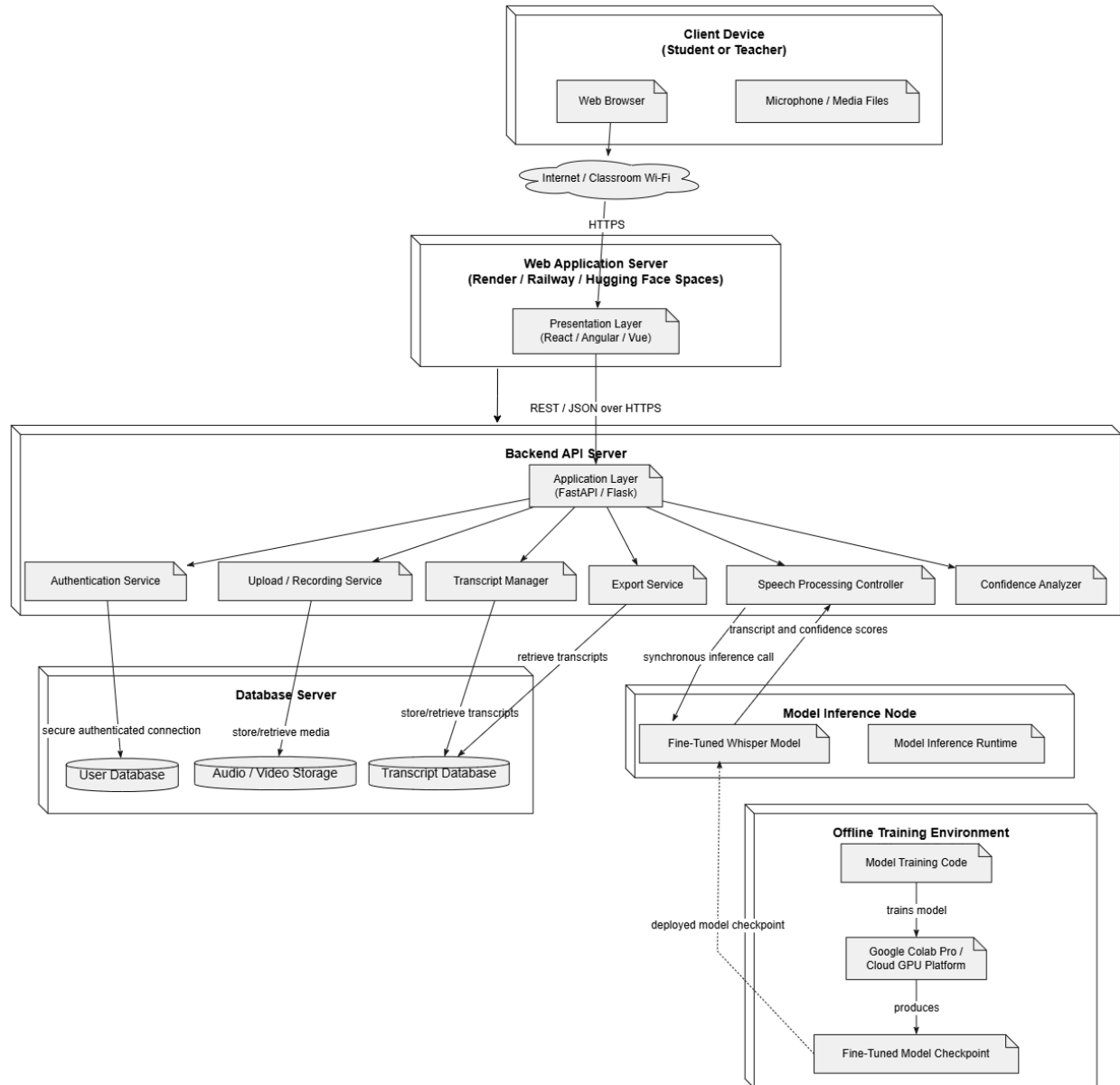


Figure 10 : Deployment View

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

8. Implementation View

8.1 Overview

The Phase 2 application's implementation model is organised into four layers: Presentation, Application, AI, and Data, plus a Users grouping representing the two actor roles who interact with the system. Each layer is a package of related subsystems, and inclusion in a layer is governed by a single rule: a component belongs to a layer if it shares that layer's primary responsibility (user interaction, business logic and coordination, speech inference, or persistent storage) and communicates with adjacent layers only through a well-defined service interface. Dependencies flow strictly downward: the Presentation Layer depends on the Application Layer, the Application Layer depends on both the AI Layer and the Data Layer, and never upward: the Data Layer has no knowledge of the Presentation Layer, and the AI Layer has no knowledge of anything above the Application Layer. This one-directional dependency rule keeps each layer independently testable and replaceable, consistent with the Modular Architecture requirement defined in the SRS.

8.2 Layers

Users

Represents the two actor roles who interact with the system: Student and Teacher. Both authenticate through the Authentication Service before reaching the Dashboard; this package contains no executable subsystems of its own and exists in the diagram purely to anchor the system's entry points.

Presentation Layer (Web Application)

Contains the Dashboard and six feature screens: Settings, Transcript Editor, Transcript Library, Lecture Captioning, Spoken Answer Transcription, and Self-Study Note Taking. This layer is responsible purely for user interaction and rendering; it holds no business logic of its own, and every action a user takes is forwarded to the Application Layer for processing.

Application Layer (Backend API)

Contains the core business logic, decomposed into six subsystems: the Authentication Service, Upload/Recording Service, Speech Processing Controller, Confidence Analyzer, Transcript Manager, and Export Service, as described in Section 5.2. This layer coordinates every request from the Presentation Layer, delegating speech inference to the AI Layer and persistence to the Data Layer.

AI Layer (Speech Recognition)

Contains a single architecturally significant subsystem: the Fine-Tuned Whisper Model produced in Phase 1. It receives audio from the Speech Processing Controller and returns transcribed text with confidence scores, forming the core intelligence of the system, and depends on nothing outside itself.

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

Data Layer

Contains three persistent stores, treated as subsystems in their own right: the Transcript Database, Audio/Video Storage, and the User Database, described further in Section 9. This layer is accessed only through Application Layer services and has no dependency on any layer above it.

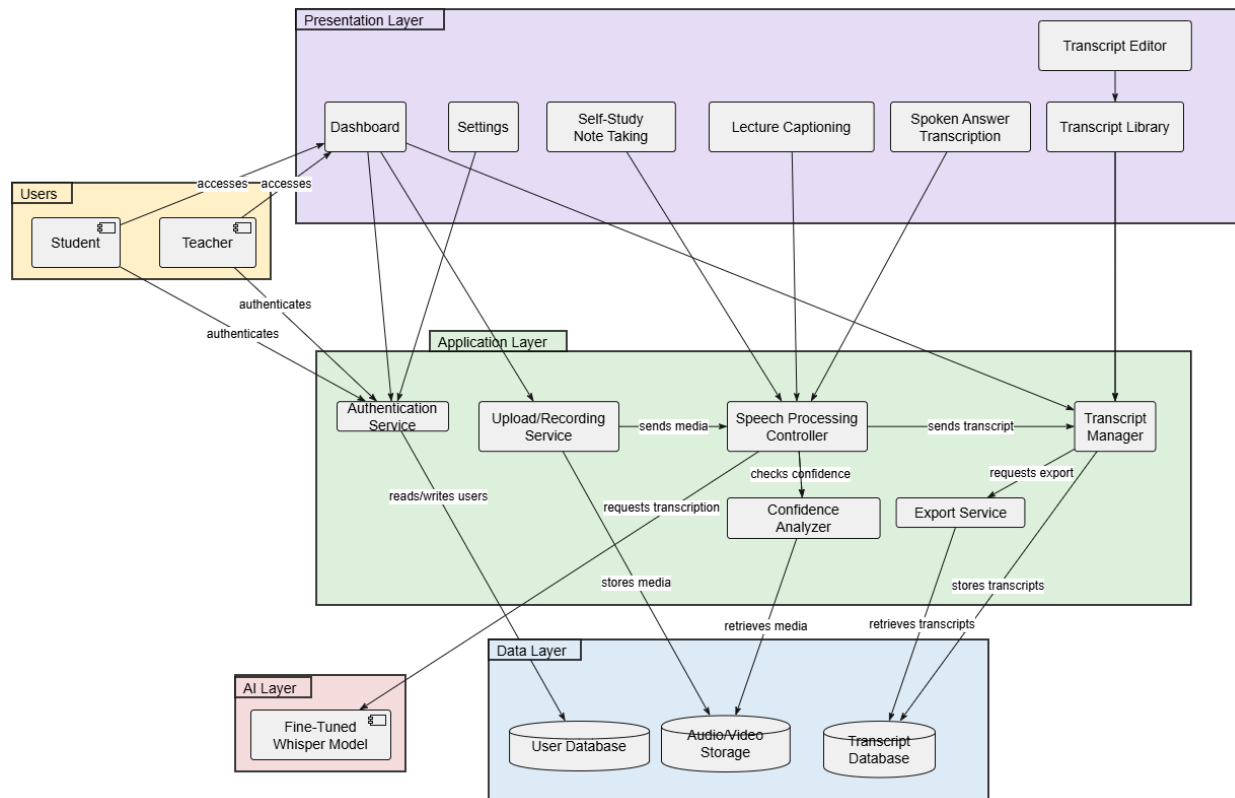


Figure 11 : Package diagram

9. Data View

Persistent data storage is architecturally significant to SinhaSpeech given the data-privacy requirements defined in the SRS. Three logical stores exist in the Data Layer:

- User Database - stores user accounts, roles (Student/Teacher), and authentication credentials.
- Audio/Video Storage - stores original lecture recordings and student-recorded audio clips, retained alongside their transcripts.
- Transcript Database - stores transcripts, transcript segments, confidence information, export records, quizzes, questions, and answer submissions. Word confidence and timing details are stored as JSON inside each transcript segment. Each answer submission links a student and question to its spoken-answer transcript.

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

All three stores are accessed only through Application Layer services, including the Authentication Service, Upload/Recording Service, Transcript Manager, Export Service, and Quiz Management Service. The Presentation Layer does not access the Data Layer directly.

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

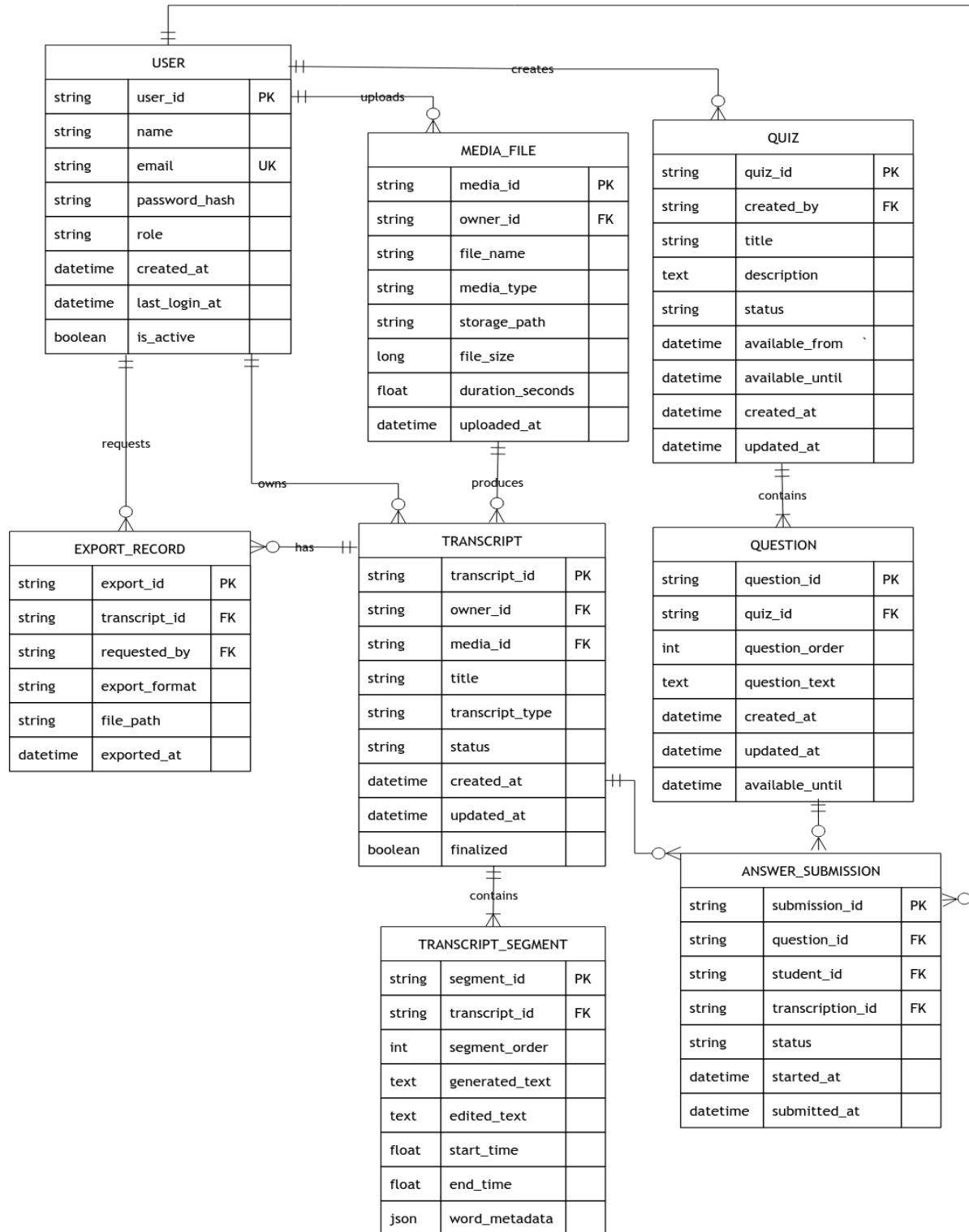


Figure 12 : Entity Relationship Diagram

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

10. Size and Performance

The Phase 1 training corpus is dominated by OpenSLR52 (approximately 224 hours of speech from 478 speakers), supplemented by several thousand utterances from YouTube-Sinhala-ASR and Biz-Brains-Academy-Sinhala, with Mozilla Common Voice Sinhala withheld for evaluation. This scale of data, together with GPU memory and Colab session limits, is the primary factor governing the choice between Whisper-Small and Whisper-Base.

For the Phase 2 application, the architecture targets the following performance and capacity characteristics, as defined in the SRS:

- Response time - uploaded-file transcription completes within a reasonable multiple of the audio's duration.
- Throughput - at least one concurrent transcription session per classroom deployment.
- Capacity - the application handles uploaded audio and video files up to a reasonable maximum size and duration without failure.
- Availability - the application is available and functional at least 95% of the time during scheduled classroom use, with an MTBF of at least 100 hours and an MTTR averaging no more than 2 hours.

11. Quality

Extensibility

The layered, modular architecture allows individual components (e.g., a new workflow, an additional export format, or a replacement inference model) to be added or modified without restructuring the rest of the system.

Reliability

The architecture separates the offline Phase 1 training environment from the live Phase 2 deployment, so instability in experimental training runs cannot affect the production application's availability.

Security and Privacy

Access to the Data Layer is restricted to authenticated requests routed through the Application Layer; classroom audio, transcripts, and student data are handled consistent with Sri Lanka's Personal Data Protection Act, with API keys and credentials managed through environment variables rather than hard-coded in source.

Portability

The Application and AI Layers are built on standard, well-documented open-source tooling (Hugging Face Transformers/Datasets, PyTorch, FastAPI/Flask), avoiding platform lock-in and allowing deployment on any

SinhaSpeech	Version : 1.0
Software Architecture Document	Date : 30-07-2026

compatible cloud host.

Maintainability

Consistent coding standards, naming conventions, and clear separation between preprocessing, fine-tuning, evaluation, and the Phase 2 application (as defined in the SRS Supportability requirements) keep the codebase approachable for future contributors.

12. References

- [1] SinhaSpeech Feasibility Study, Group 12, 26 July 2026.
- [2] SinhaSpeech Project Proposal, Group 12, 26 July 2026.
- [3] SinhaSpeech Software Requirements Specification, Version 1.0, Group 12.
- [4] O. Kjartansson, S. Sarin, K. Pipatsrisawat, M. Jansche, and L. Ha, "Crowd-Sourced Speech Corpora for Javanese, Sundanese, Sinhala, Nepali, and Bangladeshi Bengali," in Proc. 6th Workshop on Spoken Language Technologies for Under-Resourced Languages (SLTU 2018), 2018.
- [5] OpenSLR, "SLR52 – Large Sinhala ASR Training Data Set." Available: openslr.org/52 (Accessed on 25-Jul-2026).
- [6] Mozilla Common Voice – Sinhala subset. Available: mozilladatacollective.com/datasets/ (Accessed on 25-Jul-2026).
- [7] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)," OpenAI, 2022.
- [8] Hugging Face, "Fine-Tune Whisper for Multilingual ASR with Transformers." Available: huggingface.co/blog/fine-tune-whisper (Accessed on 25-Jul-2026).
- [9] SPEAK-ASR Hugging Face organization. Available: huggingface.co/SPEAK-ASR (Accessed on 25-Jul-2026).
- [10] Weights & Biases (W&B) documentation. Available: docs.wandb.ai (Accessed on 25-Jul-2026).
- [11] mermaid.js documentation, used to draw the diagrams in this document. Available: mermaid.js.org (Accessed on 26-Jul-2026).
- [12] draw.io / diagrams.net, used to render and export the diagrams in this document. Available: app.diagrams.net (Accessed on 26-Jul-2026).